

CS 3110 HW5

Name:Pulkit Saxena, NetId: Pulkit

1) Pseudocode for this question:

```
Set<Edge> NewMST(List<Edge> allEdges, Set<Edge> T, Edge e, int n, int V)
{
2)
3)   e.weight += n; //updates the weight, O(1)
4)
5)   T.remove(e); // removes the edge, O(1)
6)
7)   Set<Integer> component = dfs(T, e.u, V); //u and v (used later in the
    pseudocode) represents the endpoints of the edge e.
8)
9)   Edge minEdge = null; //O(1)
10)  int minWeight = Integer.MAX_VALUE; //O(1)
11)
12)  for (Edge edge : allEdges) { //O(E)
13)      if (edge == e) continue; //O(1)
14)      boolean crossesEdge = (component.contains(edge.u) &&
    !component.contains(edge.v)) ||
15)      (component.contains(edge.v) &&
    !component.contains(edge.u));
16)      if (crossesEdge && edge.weight < minWeight) {
17)          minEdge = edge;
18)          minWeight = edge.weight;
19)      }
20)  }
21)  if (minEdge != null && minEdge.weight < e.weight) {
22)      T.add(minEdge); //O(1)
23)  } else {
24)      T.add(e); //O(1)
25)  }
26)
27)  return T;
28) }

29) Set<Integer> dfs (Set<Edge> T, int start, int vertices) {
30)     Set<Integer> visited = new HashSet<>();
31)     Stack<Integer> stack = new Stack();
32)     Map<Integer, List<Integer>> adj = new HashMap<>();
33)
34)
35)     for (int i = 0; i < vertices; i++) //O(V)
```

```

36)     adj.put(i, new ArrayList<>());
37)   for (Edge e : T) { //O(E)
38)     adj.get(e.u).add(e.v);
39)     adj.get(e.v).add(e.u);
40)   }
41)
42)   stack.push(start);
43)   while (!stack.isEmpty()) { //O(V)
44)     int node = stack.pop();
45)     if (visited.add(node)) {
46)       for (int neighbor : adj.get(node)) {
47)         if (!visited.contains(neighbor)) {
48)           stack.push(neighbor);
49)         }
50)       }
51)     }
52)   }
53)
54)   return visited;
55) }

```

Let us analyze the runtime of this algorithm now. All the initializations, adding and removing processes have $O(1)$ time complexity. Next I have used a helper method to do the dfs algorithm which helps us to find the disconnected components of the graph after removing the edge and later on we figure out which edge can connect the disconnected components and form the new MST.

Let us take a close look at the helper method:

```

56) Set<Integer> dfs (Set<Edge> T, int start, int vertices) {
57)   Set<Integer> visited = new HashSet<>();
58)   Stack<Integer> stack = new Stack();
59)   Map<Integer, List<Integer>> adj = new HashMap<>();
60)
61)   for (int i = 0; i < vertices; i++) { //O(V)
62)     adj.put(i, new ArrayList<>());
63)   }
64)   for (Edge e : T) { //O(E)
65)     adj.get(e.u).add(e.v);
66)     adj.get(e.v).add(e.u);
67)   }
68)
69)   stack.push(start);

```

```

70) while (!stack.isEmpty()) { //O(V)
71)     int node = stack.pop();
72)     if (visited.add(node)) {
73)         for (int neighbor : adj.get(node)) { //O(E)
74)             if (!visited.contains(neighbor)) {
75)                 stack.push(neighbor);
76)             }
77)         }
78)     }
79) }
80)
81) return visited;
82) }

```

Let us take a look at the first for loop used in the method where we iterate through all the vertices:

```

for (int i = 0; i < vertices; i++) { //O(V)
83)     adj.put(i, new ArrayList<>());
84) }

```

We know that since this for loop is iterating through all the vertices of the graph its runtime must be $O(V)$.

Now let us take a look at the next for loop:

```

85) for (Edge e : T) { //O(E)
86)     adj.get(e.u).add(e.v);
87)     adj.get(e.v).add(e.u);
88) }

```

We can see that over here, this for loop iterates through all the edges of the MST and later on connects the vertices formed in the adjacency list to each other, we can say that the runtime of this algorithm is going to be $O(E)$.

Next, let us move to the while loop and analyze its runtime:

```
89) while (!stack.isEmpty()) { //O(V)
90)     int node = stack.pop();
91)     if (visited.add(node)) {
92)         for (int neighbor : adj.get(node)) { //O(E)
93)             if (!visited.contains(neighbor)) {
94)                 stack.push(neighbor);
95)             }
96)         }
97)     }
98) }
99)
100) return visited;
101) }
```

Over here we can see that the while loop runs until the stack becomes empty that means it iterates through all the vertices and hence its runtime will be $O(V)$ and since we can see that the inner for loop only iterates when the if condition is met (that is $O(1)$), we can say that the runtime of the while loop is $O(V)$.

Total runtime: $O(V + V + E) = O(V+E)$

But since this is an MST, we know that $E = V-1$, hence the total runtime becomes $O(V+V-1) = O(V)$.

Hence the runtime of the helper method is $O(V)$.

Now let us come back to the original method and look at this for loop:

```
for (Edge edge : allEdges) { //O(E)
102)     if (edge == e) continue; //O(1)
103)     boolean crossesEdge = (component.contains(edge.u) &&
    !component.contains(edge.v)) ||
104)         (component.contains(edge.v) &&
    !component.contains(edge.u));
105)     if (crossesEdge && edge.weight < minWeight) {
106)         minEdge = edge;
107)         minWeight = edge.weight;
108)     }
109) }
```

We can see that this for loop iterates through all the edges of the graph and hence we can say that the runtime of this loop is $O(E)$, since all the operations have time complexity of $O(1)$ and it is NOT iterating through a MST, where $E=V-1$. The runtime of all the operations performed after this loop is $O(1)$.

Total runtime: $O(V)$ (runtime of the helper method) + $O(E)$ (runtime of the for loop we just discussed) = $O(V+E)$

Hence, the total runtime of this algorithm is $O(V+E)$.

2)

Level	Size of the subproblem	Work per subproblem	Number of subproblems	Total work at each level
0	n	$c*n$	1	$c*n$
1	$n/5$	$c*n/5$	1	$c*n/5$
2	$n/5^2$	$c*n/5^2$	1	$c*n/5^2$
3	$n/5^3$	$c*n/5^3$	1	$c*n/5^3$
4	$n/5^4$	$c*n/5^4$	1	$c*n/5^4$
...	...			
$\log n$	$n/5^i$	1	1	1

We know that the loop stops when the size of the subproblem, that is, $n/5^i = 1$, thus $i = \log n$ for that level.

Now let us take out the total work done by summing up the work done at each level:

$$\begin{aligned}
 T(n) &= c*n + c*n/5 + c*n/5^2 + c*n/5^3 + c*n/5^4 + \dots + 1 \\
 &= c*n(1 + 1/5 + 1/5^2 + 1/5^3 + 1/5^4 + \dots)
 \end{aligned}$$

We can see that $(1 + 1/5 + 1/5^2 + 1/5^3 + 1/5^4 + \dots)$ forms a geometric series and hence we can do the rest of the calculation in the following way:

$$\begin{aligned}
 &c*n(1 + 1/5 + 1/5^2 + 1/5^3 + 1/5^4 + \dots) \\
 &= c*n(1/(1 - 1/5)) = c*n(1/(4/5)) = c*n*5/4 = O(n) \text{ (Since } c \text{ and } 5/4 \text{ are constants)}
 \end{aligned}$$

Therefore we can say that $T(n) = O(n)$, for this question.

b)

Level	Size of the subproblem	Work per subproblem	Number of subproblems	Total work at each level
0	n	n^3	1	n^3
1	n/2	$(n/2)^3$	16	$16 * (n/2)^3 = 16 * n^3 / 8 = 2 n^3$
2	n/4	$(n/4)^3$	256	$256 * (n/4)^3 = 256 * n^3 / 64 = 4 n^3$
3	n/8	$(n/8)^3$	4096	$4096 * (n/8)^3 = 4096 * n^3 / 512 = 8 n^3$
4	n/16	$(n/16)^3$	65536	$65536 * (n/16)^3 = 65536 * n^3 / 4096 = 16 n^3$
...	...			
log n	$n/2^i$	$(n/2^i)^3$	16^i	$16^i * (n/2^i)^3 = n^3 * 2^i$

We know that the function will reach its base case when at some level $n/2^i$ becomes equal to 1, hence we know that i becomes log n at that level.

Now, let us focus on calculating the runtime. Since the work done at each level is $n^3 * 2^i$, we can calculate the runtime in the following way:

$$T(n) = n^3 * \sum_{i=0}^{\log n - 1} 2^i$$

Let us consider that k = log(n) - 1, so:

$$\begin{aligned}
 T(n) &= n^3 * \sum_{i=0}^k 2^i = (2^{k+1} - 1) * n^3 \\
 &= (2^{\log n + 1 - 1} - 1) * n^3 \\
 &= (2^{\log(n)} - 1) * n^3 \\
 &= (n - 1) * n^3 = n^4 - n^3 = O(n^4)
 \end{aligned}$$

Therefore, we can say that $T(n) = O(n^4)$, for this question.

3) Let us try and do this question with the help of an example. Let us consider a grid C of 2x2 dimensions exists in this manner:

C =

5	3
2	8

Let us say that our character is placed at C(0,0) (which is 5) in this grid, so that becomes our base case as well. Let the door costs be the same as what is given to us in the question, that is, for moving in the south direction, the cost will be 3 credits, for southeast it will be 2 credits and for moving east it will be 1 credit.

Let us consider that $\text{max}[i][j]$ is a variable that stores the value of maximum credits at any point in the grid. So, that means for our base, C[0][0], $\text{max}[0][0]$ will be 5 as well.

Now, if we have to move to C(0,1) then we must move towards east from C(0,0), as that is the only way to reach over there. So, $\text{max}[0][1]$ can be calculated as follows:

$$\text{max}[0][1] = C[0][1] + (\text{max}[0][0] - 1) = 3 + (5-1) = 3 + 4 = 7$$

So our grid max looks like this as of now:

max =

5	7
---	---

Now, let us move down to C[1][0], and calculate $\text{max}[1][0]$:

$$\text{max}[1][0] = C[1][0] + (\text{max}[0][0] - 3(\text{since the cost for moving down south is 3})) = 2 + (5-3) = 2+2= 4$$

Now, if we observe that there are 3 ways to reach C[1][1] and those are as follows:

- move down south from C[0][1]
- move towards east from C[1][0]
- move towards southeast from C[0][0]

Let us calculate the value of $\text{max}[1][1]$, for each of these cases:

Case 1 (moving down south from $C[0][1]$)

$$\text{max}[1][1] = C[1][1] + (\text{max}[0][1] - 3) = 8 + (7-3) = 12$$

Case 2(moving towards east from $C[1][0]$)

$$\text{max}[1][1] = C[1][1] + (\text{max}[1][0] - 1) = 8 + (4-1) = 8 + 3 = 11$$

Case 3(moving southeast from $C[0][0]$)

$$\text{max}[1][1] = C[1][1] + (\text{max}[0][0] - 2) = 8 + (5-2) = 8+3 = 11$$

Since $12 > 11$, we know that it would be better to move down south from $C[0][1]$ as it gives a higher value of credits to move out.

Since the value of $\text{max}[1][1]$ is 12, when we move out from this grid, we know that this would be the answer for this example.

Now let us try and get the recurrence relation from what we just observed in the example above.

Let us look at how we calculated the last value, that is, $\text{max}[1][1]$. We first figured the 3 ways to reach $C[1][1]$ and then we added $C[1][1]$ to the difference of maximum credits of the previous value used in each and the cost of the way it directed us to reach $C[1][1]$, and then we took out the maximum value from them.

Observe one thing, that in all these calculations the value of $C[1][1]$, so that the value of $\text{max}[1][1]$ is only dependent on the difference of maximum credits of the previous value used in each and the cost of the way it directed us to reach $C[1][1]$, which means that for a higher difference the value of $\text{max}[1][1]$ increased, so we can determine the relation for getting the maximum number of credits as follows:

$$\begin{aligned} \text{max}[i][j] &= C[i][j] + \max \{ \\ &\text{max}[i-1][j] - 3, \text{ if } i > 0 \\ &\text{max}[i-1][j-1] - 2, \text{ if } i > 0 \text{ and } j > 0 \\ &\text{max}[i][j-1] - 1, \text{ if } j > 0 \\ &\}, \text{ where } i \text{ and } j \text{ represent the row and column numbers respectively.} \end{aligned}$$

b) Here is an efficient algorithm for this question:

```
public static int MaximumCredits(int[][] C) {  
    int M = C.length; //O(1)  
    int N = C[0].length; //O(1)  
    int[][] max = new int[M][N]; //O(1)  
  
    max[0][0] = C[0][0]; //O(1)  
  
    for (int i = 0; i < M; i++) { //O(M)  
        for (int j = 0; j < N; j++) { //O(N)  
            if (i == 0 && j == 0) continue; //O(1)  
            int Prev_val = Integer.MIN_VALUE; //O(1)  
            if (i > 0) {  
                Prev_val = Math.max(Prev_val, max[i - 1][j] - 3); //O(1)  
            }  
  
            if (j > 0) {  
                Prev_val = Math.max(Prev_val, max[i][j - 1] - 1); //O(1)  
            }  
  
            if (i > 0 && j > 0) {  
                Prev_val = Math.max(Prev_val, max[i - 1][j - 1] - 2); //O(1)  
            }  
  
            max[i][j] = C[i][j] + Prev_val; //O(1)  
        }  
    }  
  
    int result = max value that you get  
    return result;  
}
```

c) Let us analyze the worst case runtime of the algorithm that we just designed in the previous question. We know that all initializations and return statements take $O(1)$ time complexity, so that means the runtime of this algorithm basically depends on the for loop, so let us focus on that.

```
for (int i = 0; i < M; i++) { //O(M)
    for (int j = 0; j < N; j++) { //O(N)
        if (i == 0 && j == 0) continue; //O(1)
        int Prev_val = Integer.MIN_VALUE; //O(1)
        if (i > 0) {
            Prev_val = Math.max(Prev_val, max[i - 1][j] - 3); //O(1)
        }

        if (j > 0) {
            Prev_val = Math.max(Prev_val, max[i][j - 1] - 1); //O(1)
        }

        if (i > 0 && j > 0) {
            Prev_val = Math.max(Prev_val, max[i - 1][j - 1] - 2); //O(1)
        }

        max[i][j] = C[i][j] + Prev_val; //O(1)
    }
}
```

I have used a nested for loop to determine the outcome in this case. The outer loop iterates through all the rows in the grid and the inner for loop iterates through all the columns in the grid for each row. All the code that has been written inside the inner for loop has $O(1)$ time complexity, and the inner loop makes N iterations so that's why we can say that its runtime is $O(N)$, however the outer loop has M iterations, so the total runtime of this algorithm will be $O(M*N)$, since we are going through each and every in the grid and each cell is doing some constant work.

Hence, the worst case runtime of this algorithm is $O(M*N)$.